

Aprendizado por Reforço Profundo para locomoção bípede de um robô humanoide em simulação MuJoCo

Letizia L. Baptistella, Manuella F. Peres, Rafaela A. de Campos

Ciência da Computação – Centro Universitário FEI

uniflbaptistella@fei.edu.br, unifmperes@fei.edu.br, unifrcampos@fei.edu.br

Orientador, Prof. Dr. Isaac Jesus da Silva

Departamento de Ciência da Computação – Centro Universitário FEI

isaacjesus@fei.edu.br

Coorientador, Prof. Dr. Reinaldo A. C. Bianchi

Departamento de Engenharia Elétrica – Centro Universitário FEI

rbianchi@fei.edu.br

Palavras-chave, Deep Reinforcement Learning, Robótica Humanoide, Futebol de Robôs, MuJoCo, Aprendizado por Reforço

I. Resumo

A locomoção autônoma de robôs humanóides representa um dos principais desafios da robótica contemporânea. Este projeto propõe a utilização de técnicas de Aprendizado por Reforço Profundo, especificamente o algoritmo Proximal Policy Optimization (PPO), aplicadas ao modelo do robô Atom da equipe RoboFEI Humanoide em ambiente de simulação MuJoCo integrado ao servidor oficial da RoboCup 3D. O robô Atom será treinado diretamente no simulador da RoboCup 3D, categoria Adult Size, permitindo que a política aprendida seja avaliada em partidas simuladas completas contra adversários, em um contexto competitivo virtual próximo ao real. A abordagem envolve a modelagem das articulações do Atom em formato MJCF e a especificação de uma função de recompensa orientada à estabilidade e ao deslocamento eficiente. Espera-se que a política treinada produza uma locomoção bípede robusta, com baixa taxa de quedas e velocidade de caminhada e demonstrando o potencial do aprendizado por reforço como alternativa viável para a robótica humanoide competitiva.

II. Introdução

A locomoção bípede é um dos problemas mais desafiadores da robótica humanoide por exigir equilíbrio dinâmico contínuo sobre uma base de apoio muito pequena. Diferentemente de robôs com rodas ou esteiras, humanóides passam boa parte do ciclo de movimento apoiados em apenas um pé, o que aumenta significativamente a instabilidade [16]. Pequenas perturbações como irregularidades no solo ou contatos inesperados já são suficientes para causar uma queda, e isso faz com que desenvolver controladores robustos seja um requisito básico para qualquer aplicação prática, incluindo o futebol de robôs. Por essa razão, a necessidade de robustez define o ponto de partida de toda a metodologia adotada neste trabalho.

No contexto competitivo, a qualidade da locomoção é o fator mais determinante para o desempenho. MacAlpine e Stone [19] analisaram competições da RoboCup 3D e mostraram que a velocidade e a confiabilidade da caminhada são o que mais separa equipes vencedoras das demais. Um robô que cai com frequência perde posicionamento, gasta tempo em rotinas de recuperação e abre espaço para o adversário marcar, e nenhuma estratégia sofisticada compensa isso. Sendo assim, garantir uma locomoção rápida e robusta é um pré-requisito para que qualquer outra decisão de jogo tenha alguma chance de funcionar.

A abordagem baseada em *Zero Moment Point*

(ZMP) apresenta limitações relevantes nesse cenário. O método depende de modelos matemáticos precisos para manter o equilíbrio e funciona bem apenas em condições controladas [32]. Na prática, pequenas mudanças no hardware exigem recalibração manual frequente, e a adaptação a perturbações não previstas é difícil [20]. Esse conjunto de restrições faz com que o ZMP seja pouco flexível para situações imprevisíveis como as que ocorrem durante uma partida de futebol, o que motiva a busca por abordagens mais adaptativas.

Esse custo de manutenção recai de forma desigual sobre equipes universitárias. Equipes com mais estrutura conseguem manter times dedicados à recalibração do controlador, enquanto a RoboFEI precisa dividir esse tempo com outras frentes do projeto. Na prática, boa parte do esforço disponível a cada semestre acaba sendo consumida por ajustes causados por modificações no hardware em vez de ser direcionada para o desenvolvimento de novas capacidades. Esse cenário torna necessária uma abordagem que, uma vez treinada, não dependa de recalibração manual a cada mudança no sistema.

O Aprendizado por Reforço (RL) surge como alternativa adequada para lidar com essas limitações. Conforme descrito por Sutton e Barto [30], o agente aprende diretamente a partir da interação com o ambiente, ajustando seu comportamento com base em recompensas sem depender de um modelo explícito

do sistema. Com isso, o RL reduz a necessidade de ajustes frequentes e se mostra mais adequado para ambientes dinâmicos como o futebol de robôs.

O objetivo principal deste trabalho é desenvolver e avaliar uma política de locomoção bípede para o robô Atom da RoboFEI usando Aprendizado por Reforço Profundo, especificamente o algoritmo Proximal Policy Optimization (PPO) [27], treinado e avaliado no simulador MuJoCo [31] integrado ao ambiente oficial da RoboCup 3D. Como objetivo secundário, o trabalho documenta o processo de modelagem do Atom em MJCF e sua integração com o servidor de simulação, gerando uma base técnica reutilizável para projetos futuros da equipe. O trabalho se insere no contexto da RoboCup [15], competição criada em 1997 com a meta de que humanoides autônomos vençam a seleção humana campeã do mundo até meados deste século, e representa um passo importante da RoboFEI nessa direção.

O restante do artigo está organizado da seguinte forma. A Seção II revisa a literatura sobre fundamentos de RL e DRL, algoritmos para controle contínuo e aplicações em locomoção humanoide e futebol de robôs. A Seção III descreve a metodologia, incluindo a modelagem do ambiente, os espaços de observação e ação, a arquitetura da rede neural e a função de recompensa. A Seção IV apresenta os resultados preliminares obtidos até agora. A Seção V discute as decisões tomadas, limitações e próximos passos. A Seção VI traz o cronograma das etapas restantes.

III. Revisão Bibliográfica

A revisão bibliográfica foi organizada em três tópicos, sendo eles os fundamentos teóricos do RL e DRL, os algoritmos para controle contínuo e as aplicações em locomoção humanoide e futebol de robôs. Cada trabalho revisado influenciou diretamente alguma escolha do projeto, e o objetivo desta seção é mostrar como a literatura sustenta tanto a viabilidade quanto a relevância do trabalho. O DRL oferece muitas variações possíveis, e a escolha errada de algoritmo ou função de recompensa pode comprometer meses de treinamento. Por isso, a revisão não é apenas contextualização, mas o fundamento que justifica cada opção adotada.

A. Fundamentos de RL e DRL

O fundamento central do RL é o Processo de Decisão de Markov (MDP), que define como um agente aprende a partir da interação com o ambiente. Em um MDP, o agente observa o estado s , escolhe uma ação a , recebe uma recompensa r e transita para um novo estado s' , com o objetivo de aprender uma política π que maximize a soma acumulada de recompensas [30]. No caso do Atom, o estado corresponde ao vetor de observações extraído da simulação, composto pelas posições e velocidades generalizadas do robô, a ação corresponde a comandos contínuos enviados aos

atuadores de posição das articulações, e a recompensa incentiva o avanço na direção definida como frontal, mantendo estabilidade e penalizando quedas ou comandos bruscos. Compreender o MDP foi importante para deixar claro por que a função de recompensa tem tanto impacto no resultado final, assunto retomado na Seção IV.

A função valor-ação $Q^\pi(s, a)$ é a ferramenta matemática que permite ao agente avaliar o quanto cada decisão contribui para o retorno futuro. Ela quantifica o retorno esperado ao executar a ação a no estado s e seguir a política π a partir daí [30], [33]:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s, a_0 = a \right] \quad (1)$$

onde s_0 é o estado inicial, a_0 é a ação inicial, r_{t+1} é a recompensa recebida no passo $t+1$, e $\gamma \in [0, 1]$ é o fator de desconto que controla o quanto as recompensas futuras influenciam as decisões presentes. Em locomoção, comportamentos como manter o equilíbrio se manifestam ao longo de dezenas de passos, de forma que valores altos de γ são necessários para que o agente seja incentivado a sustentar posturas estáveis durante uma passada completa. Sem isso, o agente tenderia a otimizar apenas o instante atual, produzindo políticas que parecem funcionar no curto prazo mas colapsam em sequências mais longas. O valor exato de γ será definido durante os experimentos iniciais com o Atom.

A equação de Bellman estabelece uma relação recursiva que torna o aprendizado iterativo possível, permitindo que o agente se atualize passo a passo sem esperar o fim do episódio [30]:

$$Q^\pi(s, a) = \mathbb{E} \left[r + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s', a') \right] \quad (2)$$

onde r é a recompensa imediata, s' é o estado seguinte, a' é uma ação possível em s' , e $\pi(a'|s')$ é a probabilidade de escolher a' segundo a política π . Quando a estimativa do valor do próximo estado é imprecisa nas fases iniciais do treinamento, as atualizações ficam instáveis e o agente pode desaprender comportamentos que levou horas para adquirir, fenômeno conhecido como *catastrophic forgetting* [22]. Em locomoção bípede isso é problemático porque o agente pode investir muitas interações aprendendo a se manter de pé e perder esse comportamento após uma sequência de atualizações ruins. Esse risco foi levado em conta na definição do tamanho dos lotes de atualização e da frequência de avaliação durante o treinamento do Atom.

O DQN (*Deep Q-Network*), proposto por Mnih et al. [22], estabeleceu as bases do DRL moderno ao demonstrar que redes neurais profundas conseguem aproximar funções de valor em espaços de estado complexos.

O algoritmo combina Q-Learning clássico [33] com duas inovações que estabilizam o treinamento, sendo elas uma memória de experiências (*replay buffer*), que armazena transições passadas e permite reamostrar de forma decorrelacionada, e uma rede-alvo separada, que mantém valores de referência estáveis por vários passos. No experimento seminal, o agente aprendeu a jogar dezenas de jogos de Atari acima do nível humano usando apenas pixels como entrada, o que valida o uso de DRL para lidar com vetores de alta dimensão como o que descreve o estado do Atom. A limitação crítica do DQN, porém, é que ele só funciona com espaços de ações discretos, já que a maximização de Q se torna intratável quando as ações são contínuas como no controle por torque das articulações, e por isso foi necessário recorrer a algoritmos específicos como o PPO.

B. Algoritmos para Controle Contínuo

O controle de articulações por torque exige um algoritmo capaz de lidar com espaço de ações contínuo, o que elimina o DQN como opção. O PPO foi escolhido como algoritmo principal por apresentar bom equilíbrio entre estabilidade e desempenho em tarefas de controle contínuo. Schulman et al. [27] demonstram que o uso de uma função objetivo com *clipping* limita atualizações bruscas na política, reduzindo instabilidades durante o treinamento. Em locomoção bípede, onde pequenas variações podem levar a quedas frequentes, essa estabilidade é tão importante quanto a qualidade final da política aprendida, e por isso o PPO se mostrou a escolha mais adequada para o controle do Atom. A fórmula utilizada no PPO pode ser vista abaixo:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (3)$$

onde θ são os parâmetros da política, $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ é a razão entre as probabilidades da ação sob a nova e a antiga política, ϵ é o hiperparâmetro de *clipping* que limita o tamanho da atualização, e $\hat{A}_t = Q^\pi(s_t, a_t) - V^\pi(s_t)$ é a estimativa de vantagem, que indica se a ação tomada foi melhor ou pior do que o valor esperado para aquele estado, sendo $V^\pi(s_t)$ a função de valor que estima o retorno médio a partir de s_t [30]. Quando $r_t(\theta)$ sai do intervalo $[1-\epsilon, 1+\epsilon]$, o gradiente é cortado e a atualização é descartada nessa direção, impedindo mudanças desestabilizadoras sem o custo de resolver um problema de otimização restrita a cada passo como ocorre no TRPO [26]. Isso torna o PPO mais simples de implementar e ajustar, o que é uma vantagem dentro do cronograma disponível.

Os demais algoritmos para controle contínuo foram avaliados e descartados por razões específicas. O SAC [12] adiciona um termo de entropia ao objetivo para es-

timular a exploração, mas possui mais hiperparâmetros para calibrar, e o coeficiente de temperatura da entropia é sensível justamente quando o agente ainda está aprendendo comportamentos básicos como não cair. O TD3 [7] usa dois críticos independentes para reduzir a superestimação do valor de ação, mas sua documentação em locomoção bípede é bem menos extensa do que a do PPO. O DDPG [18], antecessor do TD3, foi descartado pelos mesmos motivos e pela instabilidade de treinamento conhecida em dinâmicas complexas como a de um humanoide. Diante dessas comparações, o PPO se mostrou o ponto de partida mais adequado para o Atom, ficando os demais algoritmos reservados para extensões futuras.

A escolha do PPO também é sustentada por evidências sistemáticas na literatura de controle motor simulado. Duan et al. [6] compararam diversos algoritmos em tarefas de controle motor e confirmaram que o PPO apresenta bom custo-benefício entre estabilidade e desempenho em sistemas com muitos graus de liberdade, que é exatamente o perfil do Atom. Ainda assim, Kober et al. [16] alertam que o maior desafio do RL em robótica não está no algoritmo em si, mas no design da função de recompensa, assunto tratado em detalhe na Seção IV. A escolha do PPO é, portanto, apenas o primeiro passo de um processo metodológico mais amplo.

C. Locomoção Humanoide e Futebol de Robôs

O DRL se mostra viável em robótica com muitos graus de liberdade mesmo sem um modelo explícito do ambiente. Peters et al. [25] mostraram que algoritmos de gradiente de política funcionam em sistemas físicos reais, abrindo caminho para uma linha de pesquisa que hoje apresenta resultados relevantes em diversas plataformas robóticas. A partir daí, Heess et al. [13] demonstraram que comportamentos locomotores complexos podem surgir a partir de objetivos simples de treinamento, como avançar sem cair, sem que o projetista precise especificar manualmente como cada perna deve se mover. Para o Atom, isso tem uma implicação prática direta, pois o próprio algoritmo pode descobrir padrões eficientes de movimento e reduzir um dos principais gargalos dos controladores baseados em ZMP.

Em controladores ZMP, ações como levantar o pé, transferir peso ou coordenar os braços para manter o equilíbrio precisam ser especificadas e ajustadas manualmente, o que demanda tempo e conhecimento especializado. Com DRL, como mostram Heess et al. [13], esses comportamentos podem emergir durante o treinamento de forma conjunta. No caso do Atom, que possui 6 graus de liberdade nos braços além dos 12 nas pernas, isso é útil porque abre espaço para explorar mais possibilidades de movimento em vez de manter os braços fixos. O DRL não apenas substitui o ZMP como também permite comportamentos mais ricos do que os possíveis com programação manual.

A transferência de políticas treinadas em simulação

para robôs reais ainda é um desafio conhecido, mas existem estratégias consolidadas para reduzi-lo. Haarnoja et al. [12] e Hwangbo et al. [14] apresentaram resultados positivos de locomoção aprendida por DRL em plataformas físicas, sendo que Hwangbo et al. modelaram os atuadores com redes neurais em vez de assumir motores ideais, o que melhorou significativamente a transferência da política. Essa abordagem é relevante para o Atom porque os servomotores Dynamixel XM540 e XM430 possuem características de resposta que nem sempre são bem representadas por modelos simplificados, principalmente sob cargas maiores durante a fase de apoio simples. A transferência para o hardware físico está fora do escopo deste trabalho, mas a infraestrutura será construída visando facilitar essa implementação no futuro.

As variáveis incluídas no vetor de observações foram escolhidas com base em trabalhos que investigaram quais informações são mais relevantes para manter o equilíbrio em humanoides. Ha et al. [11] apresentam um *survey* recente sobre quais tipos de sensores são mais úteis nesse contexto, e Shi et al. [28] mostraram que incluir sinais de contato dos pés e velocidade angular do tronco melhora a estabilidade da política aprendida. Melo et al. [21] e Gonçalves [9] observaram que políticas treinadas com DRL lidam melhor com colisões e perturbações externas do que controladores baseados em ZMP, o que indica que a abordagem é adequada para o ambiente competitivo da RoboCup. Esses resultados justificam diretamente as variáveis escolhidas para o vetor de observações descrito na Seção IV.

A reprodutibilidade dos resultados é um ponto de atenção em DRL que precisa ser tratado desde o início. Duan et al. [6] observaram que pequenas diferenças na configuração do treinamento, como taxa de aprendizado, tamanho do minibatch ou limites do espaço de ações, podem levar a resultados bastante diferentes mesmo com o mesmo algoritmo e tarefa. Configurações ligeiramente distintas podem tornar comparações entre trabalhos injustas, o que dificulta avaliar o real progresso da área. Por esse motivo, todos os hiperparâmetros utilizados neste trabalho serão registrados e disponibilizados junto com o código, de modo que os resultados possam ser reproduzidos por outras equipes.

No contexto específico do futebol de robôs, os trabalhos de Abreu et al. [1], [2] são diretamente relevantes para este projeto. O primeiro mostra que é possível aprender a caminhar e chutar por DRL sem movimentos programados manualmente, usando a simulação da RoboCup 3D, enquanto o segundo apresenta políticas com desempenho comparável ao de equipes que desenvolveram seus controladores manualmente ao longo de anos. Maximo e Ribeiro [20] apontam que controladores ZMP utilizados por equipes brasileiras exigem recalibração frequente e apresentam limitações sob perturbações externas, o que reforça a motivação para investigar o DRL como alternativa concreta. Para a

RoboFEI, esses resultados em conjunto sugerem que o investimento em DRL pode trazer ganhos competitivos reais, e não apenas contribuições teóricas.

Um trabalho de referência direta para este projeto é o de Gianzanti [8], ex-aluno de mestrado do Centro Universitário FEI, cujo estudo investigou o uso de DRL com PPO para controle de locomoção do robô Darwin OP3. O trabalho resultou em dois repositórios públicos no GitHub, sendo o primeiro o *op3_model*, que implementa um ambiente compatível com o OpenAI Gym [4] integrando o Darwin OP3 ao MuJoCo [31], e o segundo o *op3_trainner*, que contém o notebook de treinamento com PPO. Embora o Darwin OP3 seja diferente do Atom, as decisões documentadas nesses repositórios sobre definição do vetor de observações, formato do espaço de ações e estrutura da função de recompensa servem como ponto de partida mais informado do que partir exclusivamente da literatura geral. As lições sobre instabilidades de treinamento e sensibilidade dos hiperparâmetros serão diretamente consideradas na configuração dos experimentos com o Atom, tornando esses repositórios uma referência técnica de grande valor para o projeto. O MuJoCo [31] foi escolhido principalmente por sua ampla utilização em pesquisas de locomoção e pela boa fidelidade na simulação de contatos entre corpos rígidos.

IV. Metodologia

A metodologia foi dividida em três etapas, sendo elas o levantamento bibliográfico, a modelagem do ambiente de simulação com o robô Atom e o treinamento e avaliação do agente. Cada etapa produz conteúdos importantes para a seguinte, e iniciar o treinamento antes de validar o modelo físico produziria diagnósticos equivocados difíceis de corrigir. A segunda etapa está em estágio avançado, e a terceira começa em paralelo à medida que partes do modelo ficam disponíveis para testes, o que foi uma decisão visando aproveitar o tempo disponível sem comprometer a qualidade dos resultados.

A Figura 1 resume o processo completo desde a modelagem no Onshape até a avaliação final em partidas da RoboCup 3D, mostrando as dependências entre as etapas.

A. Modelagem do Ambiente de Simulação

A escolha do simulador foi uma das primeiras decisões técnicas do projeto e influencia diretamente todo o restante do processo. O MuJoCo [31] foi escolhido principalmente pelo uso de um integrador numérico especializado para corpos rígidos com restrições de contato, que permite simular a dinâmica de um humanoide de forma relativamente estável sem exigir hardware muito potente. Em simuladores menos robustos, inconsistências nas forças de contato podem surgir quando o robô apoia o pé no solo ou sofre uma queda, levando o agente a explorar artefatos numéricos da simulação em vez de aprender um padrão real de locomoção [5]. Esse

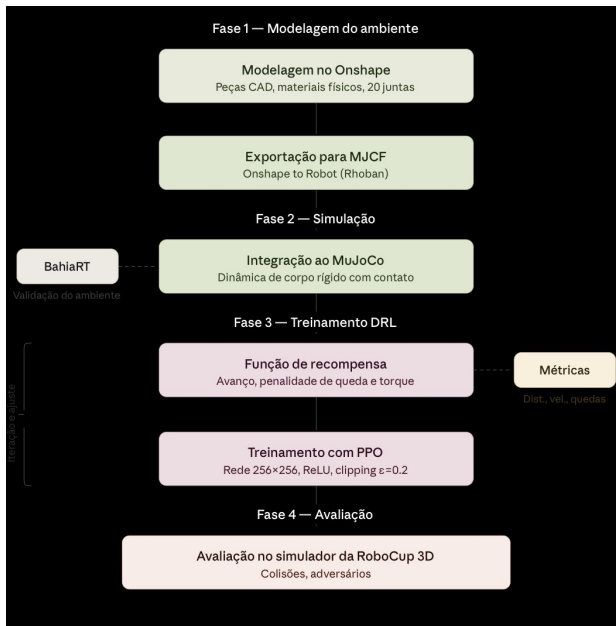


Figure 1: Fluxograma do processo: modelagem no Onshape, exportação MJCF, treinamento com PPO e avaliação em partidas da RoboCup 3D.

tipo de problema tornaria qualquer política aprendida inútil ao ser testada no ambiente real, e é por isso que a escolha do simulador é determinante para a validade de todos os resultados posteriores.

Usar o simulador oficial da RoboCup 3D como ambiente de treinamento também foi uma decisão natural, em grande parte pelo conhecimento prévio das integrantes da RoboFEI com essa infraestrutura. Uma vantagem dessa escolha é que o agente aprende diretamente no ambiente onde irá competir, incluindo o atrito característico da grama sintética, as dimensões reais do campo e os estados de jogo típicos de uma partida. Isso reduz o risco de que uma política treinada em um ambiente simplificado apresente comportamento inesperado durante a competição, que é exatamente o tipo de falha mais difícil de diagnosticar e corrigir próximo a um evento.

O ambiente utilizado é o *rcssservermj*, servidor oficial da RoboCup 3D baseado em MuJoCo, que funciona em uma arquitetura cliente-servidor na qual o servidor é iniciado com a configuração do campo Adult Size e os agentes se conectam por meio de soquetes TCP informando parâmetros como endereço, porta, número do jogador e nome do time. Essa arquitetura facilita a execução de vários agentes simultaneamente, e no caso deste projeto permitiu paralelizar o treinamento com oito ambientes ao mesmo tempo sem qualquer modificação no servidor. Essa paralelização foi essencial para viabilizar o volume de treinamento necessário dentro do prazo disponível.

A arquitetura cliente-servidor também ajuda a separar as etapas de treinamento e avaliação, o que nem sempre é feito em projetos de DRL. Durante o treinamento, os agentes utilizam uma política em modo de exploração, com perturbações aleatórias nas ações para

incentivar a exploração de novas regiões do espaço de estados. Na avaliação, a política é executada para medir o desempenho real. Adotar essa separação desde o início evita confusões e retrabalho nas fases mais avançadas do treinamento, e por isso foi importante definir esse protocolo antes mesmo de iniciar os primeiros experimentos.

Como validação inicial da infraestrutura, o ambiente foi testado com o modelo Booster T1 da equipe BahiaRT [29], escolhido por ser amplamente utilizado por outros times em competições internacionais na categoria Adult Size. A ideia foi garantir que eventuais problemas pudessem ser atribuídos ao ambiente ou à configuração do treinamento, e não ao modelo do robô em si. Todos os experimentos executados com esse modelo ocorreram de forma estável e sem instabilidades numéricas perceptíveis, confirmando que a infraestrutura estava funcionando corretamente com diferentes geometrias e dinâmicas. Com essa etapa concluída, foi possível avançar para a modelagem do próprio Atom com mais confiança na infraestrutura.

Modelagem do Atom no Onshape

Modelar o Atom no MuJoCo exigiu um conjunto específico de ferramentas para garantir que o modelo físico fosse preciso o suficiente para o treinamento. O ponto de partida foram as peças modeladas pela equipe de mecânica no Autodesk Fusion, que foram exportadas e importadas para o Onshape, plataforma CAD baseada em nuvem. A migração para o Onshape foi motivada pela disponibilidade da ferramenta *Onshape to Robot*, desenvolvida pela equipe Rhoban [3], [10], que automatiza a exportação do modelo para o formato MJCF nativo do MuJoCo. Construir esse arquivo manualmente para um modelo com 20 juntas seria longo e sujeito a erros difíceis de detectar sem análise cuidadosa da dinâmica, de forma que a automação foi um pré-requisito para viabilizar a modelagem dentro do cronograma. No Onshape, foram realizadas a montagem completa das peças, a atribuição de materiais e a definição das juntas cinemáticas, conforme apresentado na Tabela 1.

Table 1: Distribuição das 20 juntas do robô Atom

Região	Juntas	Movimentos principais
Perna esquerda	6	Quadril, joelho, tornozelo
Perna direita	6	Quadril, joelho, tornozelo
Braço esquerdo	3	Ombro, cotovelo
Braço direito	3	Ombro, cotovelo
Cabeça	2	Pan, tilt
Total	20	—

Os limites mecânicos das articulações do quadril e do tornozelo definem até onde cada junta pode girar no

simulador, e sua correta especificação é um requisito crítico para a validade da política aprendida. Valores incorretos fariam a política adotar posturas que o hardware real não consegue replicar, tornando inúteis os resultados obtidos em simulação. Isso é bastante sensível nas articulações do tornozelo, que operam próximas ao limite mecânico durante a fase de apoio simples e são responsáveis por boa parte da correção de equilíbrio lateral. A definição cuidadosa desses limites é, portanto, uma condição necessária para que a política aprendida seja transferível ao hardware.

A definição das geometrias de colisão foi outro desafio importante na modelagem, pois determina diretamente a fidelidade da dinâmica de contato simulada. O MuJoCo não usa a geometria visual das peças para calcular colisões, e cada corpo rígido precisa de uma geometria simplificada, tipicamente caixas, cilindros ou cápsulas, que aproxime a forma real com custo computacional reduzido [14]. Uma geometria muito grosseira pode fazer o robô deslizar sobre o solo de forma não realista, enquanto uma muito detalhada aumenta o tempo de simulação a ponto de inviabilizar o treinamento dentro do prazo. O equilíbrio adotado priorizou a fidelidade nas regiões de contato mais frequentes durante a caminhada, principalmente solas dos pés e joelhos, e foi validado visualmente antes de qualquer treinamento.

A atribuição de materiais é igualmente importante porque o MuJoCo usa as propriedades físicas de cada peça para calcular a dinâmica do sistema. Densidades incorretas deslocariam o centro de massa para uma posição diferente da real, fazendo o agente aprender a compensar desequilíbrios que simplesmente não existem no robô físico [14]. Para as peças fabricadas por impressão 3D, foram usados materiais da biblioteca do Onshape com base nas especificações de fabricação. Cada valor foi revisado individualmente antes da exportação, e a Tabela 2 resume os materiais e densidades atribuídos a cada grupo de componentes.

Table 2: Materiais atribuídos às peças do Atom no Onshape

Componente	Material	Dens. (lb/in ³)
Tampa peito, porta bateria, puxador	PLA	0,045
Capacete, amortecedores	ABS	0,038
NUC (computador)	Acrílico	0,042
Bateria	Teflon	0,078
Motores XM540/XM430	Fluoroelast.	0,065
Peças estruturais	Al. 6061	0,098

Três componentes têm grande influência na dinâmica do sistema e merecem atenção especial. A bateria, representada pelo Teflon (0,078 lb/in³), pos-

sui densidade consideravelmente maior que as peças plásticas e está posicionada no tronco, afetando diretamente a altura do centro de massa. Os motores Dynamixel, representados pelo Fluoroelastômero (0,065 lb/in³), estão distribuídos por todas as articulações e somam uma parcela relevante da massa total. O computador NUC, representado pelo Acrílico (0,042 lb/in³), concentra massa significativa na região central do tronco e influencia a inércia rotacional do robô em torno dos eixos sagital e frontal, afetando a dinâmica de equilíbrio durante as fases de apoio simples. Qualquer erro na densidade de um desses componentes poderia fazer o agente compensar desequilíbrios inexistentes no robô real, e por isso cada valor foi revisado antes de exportar o modelo. A Figura 2 apresenta a visualização do centro de massa calculado a partir dessas densidades, confirmando que sua posição está coerente com o esperado para o robô físico.

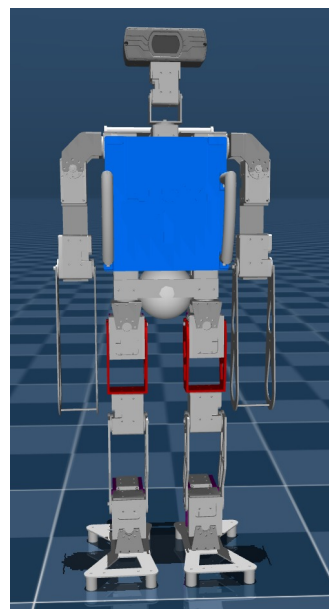


Figure 2: Visualização do centro de massa do robô Atom no Onshape, calculado a partir das densidades atribuídas a cada componente.

Com a montagem, os materiais e as juntas definidos, a ferramenta *Onshape to Robot* foi usada para exportar o modelo em formato MJCF, arquivo que descreve toda a estrutura do robô para o MuJoCo, incluindo geometria, massas, momentos de inércia, limites das juntas e relações cinemáticas entre os corpos. A correta identificação de cada grau de liberdade no modelo é uma condição necessária para que os torques sejam aplicados nas articulações certas durante o treinamento, pois uma inversão entre DOFs simétricos como o *yaw* e o *roll* do quadril produziria comandos fisicamente incoerentes e inviabilizaria a convergência da política. A Figura 3 apresenta o diagrama do robô Atom com as 20 articulações numeradas em sua localização física, e a tabela 3 apresenta a tabela de correspondência entre esses índices e os nomes dos *dofs* utilizados no modelo

MJCF, cobrindo desde os graus da cabeça nos índices 1 e 2 até a perna direita completa nos índices 12 a 20, passando pelo braço esquerdo nos índices 3 a 5 e pela perna esquerda nos índices 6 a 11. Essa numeração é diretamente consistente com o vetor de 52 observações descrito na Tabela 4, de modo que os ângulos e velocidades de cada articulação ocupam sempre a mesma posição no vetor de entrada da rede, eliminando uma fonte de erro silenciosa que seria difícil de diagnosticar após o início do treinamento.

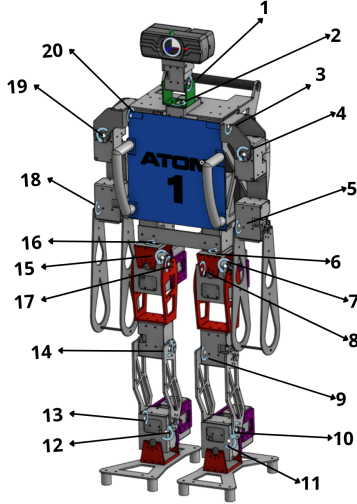


Figure 3: Diagrama do robô Atom com as 20 articulações numeradas de 1 a 20, identificando a localização física de cada grau de liberdade no modelo MJCF.

1	dof_head_tilt	11	dof_left_foot_roll
2	dof_head_pan	12	dof_right_foot_roll
3	dof_left_shoulder_pitch	13	dof_right_foot_pitch
4	dof_left_shoulder_roll	14	dof_right_knee
5	dof_left_elbow_pitch	15	dof_right_leg_pitch
6	dof_left_leg_yaw	16	dof_right_leg_roll
7	dof_left_leg_roll	17	dof_right_leg_yaw
8	dof_left_leg_pitch	18	dof_right_elbow_pitch
9	dof_left_knee	19	dof_right_shoulder_roll
10	dof_left_foot_pitch	20	dof_right_shoulder_pitch

Table 3: Tabela de correspondência entre os índices numéricos das articulações e os nomes dos graus de liberdade (*dofs*) utilizados no modelo MJCF do Atom. A numeração é consistente com o diagrama da Figura 3 e com o vetor de 52 observações descrito na Tabela 4.

Espaços de Observações e Ações

O vetor de observações do agente foi definido para oferecer ao controlador informação suficiente sobre a postura atual e a tendência de movimento do robô, sem variáveis redundantes. Com base em Haarnoja et al. [12], Hwangbo et al. [14], Ha et al. [11] e Shi et al. [28], o vetor inclui ângulos e velocidades das 20 articulações, velocidade linear e angular do centro de massa, orientação do tronco e sinais de contato dos pés com o solo, conforme a Tabela 4. Variáveis redun-

dantes aumentariam desnecessariamente a dimensão do espaço de estado e prejudicariam a convergência, e por isso cada variável incluída tem um papel funcional específico no controle.

Table 4: Vetor de observações planejado para o agente

Variável	Dim.	Papel no controle
Ângulos das articulações	20	Postura atual
Velocidades das articulações	20	Dinâmica da postura
Velocidade linear do CoM	3	Estado global
Velocidade angular do CoM	3	Rotação do tronco
Orientação (quaternion)	4	Inclinação/equilíbrio
Contato pé esquerdo	1	Apoio no solo
Contato pé direito	1	Apoio no solo
Total	52	–

O espaço de ações é composto pelos torques aplicados nas 20 articulações, com valores normalizados para $[-1, 1]$ e reescalados para os limites físicos de cada junta. Essa normalização é necessária porque redes neurais têm dificuldade em lidar com saídas em escalas muito diferentes entre articulações. Sem ela, articulações com limites de torque muito distintos, como quadril e tornozelo, produziram gradientes em escalas incompatíveis e desestabilizariam o treinamento desde as primeiras iterações [18].

B. Treinamento e Avaliação do Agente

Arquitetura da Rede Neural

A rede neural que representa a política do agente tem papel central no DRL. Uma rede neural artificial é um modelo computacional composto por camadas de unidades chamadas neurônios, que transformam progressivamente a entrada em uma saída útil. No contexto do PPO aplicado à locomoção, a rede recebe o vetor de observações do robô como entrada, composto pelas posições generalizadas (qpos) e velocidades generalizadas (qvel) do modelo na simulação, processa essa informação em camadas intermediárias e produz como saída ações contínuas no intervalo $[-1, 1]$, uma para cada atuador do robô. Essas ações são multiplicadas por uma escala e somadas ao controle inicial, formando a referência de posição final enviada a cada junta. Os atuadores definidos no modelo MuJoCo são atuadores de posição com ganho proporcional e amortecimento, de forma que o agente não envia torque diretamente às articulações, mas define referências de posição, e o simulador calcula a resposta física resultante.

A arquitetura foi definida para equilibrar capacidade representacional e eficiência computacional den-

tro do hardware disponível. A rede é composta por duas camadas densas com 256 neurônios cada e ativação ReLU, configuração amplamente usada em estudos de locomoção bípede com DRL [6], [12]. A função ReLU é definida como $f(x) = \max(0, x)$ e é preferida por ser computacionalmente barata e por não sofrer do problema de gradiente desaparecente de forma tão acentuada quanto funções como a sigmoide, o que torna o treinamento mais estável em redes com múltiplas camadas [17]. Com menos neurônios, o modelo pode não ter capacidade suficiente para representar a política de locomoção, e com mais a convergência fica mais lenta sem ganho proporcional de desempenho, o que justifica a configuração escolhida.

A escolha dessa arquitetura também é amparada pela experiência anterior com o mesmo simulador. Ginzanti [8] utilizou uma rede com estrutura semelhante para o Darwin OP3 no MuJoCo, e os resultados parciais documentados nos repositórios indicam que essa configuração é suficiente para representar políticas de locomoção em humanoides de porte similar ao Atom. Isso reduz o risco de que a arquitetura seja um problema durante o treinamento, e assim o esforço de ajuste pode ser concentrado na função de recompensa e nos hiperparâmetros do PPO, que são os componentes de maior impacto no resultado final.

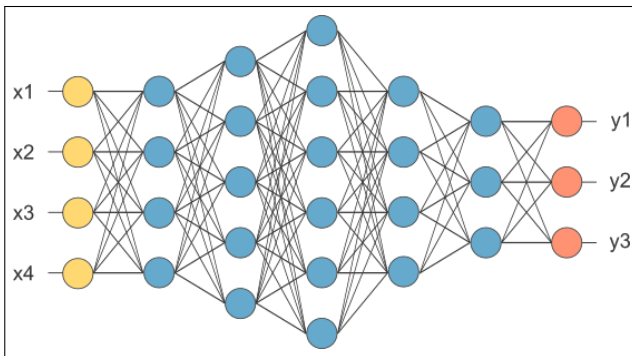


Figure 4: Exemplo ilustrativo de rede neural densa multicamada. A figura serve apenas como referência visual do conceito, a arquitetura utilizada neste trabalho possui camada de entrada de dimensão 52, duas camadas ocultas de 256 neurônios com ativação ReLU e camada de saída com 20 ações contínuas normalizadas. Fonte: [23].

A Figura 4 apresenta um exemplo ilustrativo de rede densa para fins didáticos, referência visual para a arquitetura efetivamente adotada descrita a seguir.

O embaralhamento das experiências antes das atualizações do PPO tem um papel relevante na estabilidade do treinamento. Quando o agente coleta dados em vários ambientes paralelos, todas essas experiências são reunidas em um único buffer, e antes de atualizar o modelo elas são embaralhadas e divididas em minibatches. Isso evita que o modelo aprenda com amostras muito correlacionadas em sequência, pois se as experiências fossem usadas na ordem em que foram coletadas, o gradiente poderia ser puxado repetida-

mente para a mesma direção, tornando o treinamento instável ou enviesado. Com o embaralhamento, cada minibatch contém uma mistura de experiências de diferentes ambientes e momentos das trajetórias, o que melhora o equilíbrio e a capacidade de generalização do agente, aspecto importante quando os oito ambientes paralelos do Atom evoluem de formas distintas ao longo do treinamento.

Os hiperparâmetros do PPO serão definidos com base nos valores padrão reportados em Schulman et al. [27] e ajustados a partir das observações dos treinamentos práticos. Isso reduz o número de decisões arbitrárias no início do treinamento e fornece um ponto de partida mais informado do que simplesmente usar os padrões da biblioteca. A estratégia adotada será inserir o modelo MJCF do Atom diretamente no código de Ginzanti [8], substituindo o Darwin OP3, de forma a reaproveitar a infraestrutura já validada antes de migrar o agente para o ambiente oficial da RoboCup 3D, o que permite isolar eventuais problemas de integração do modelo em um ambiente mais controlado.

Para validar a infraestrutura de treinamento antes da integração completa do Atom ao ambiente da RoboCup 3D, foram conduzidos 10 experimentos exploratórios com PPO, variando hiperparâmetros como `n_steps`, `n_epochs` e `batch_size`. Nesses experimentos, a rede neural adotou uma arquitetura Actor-Critic com política `MlpPolicy`, em que o agente recebe 53 valores de observação como entrada, e tanto o *actor* quanto o *critic* utilizam duas camadas ocultas com 64 neurônios cada e ativação Tanh. O *actor* produz 20 ações contínuas normalizadas no intervalo $[-1, 1]$, enquanto o *critic* estima um escalar de valor para o estado atual. O total de parâmetros treináveis é de 16.617, distribuídos igualmente entre as duas redes. A Tabela 5 apresenta a configuração base utilizada nos experimentos.

Table 5: Hiperparâmetros base dos experimentos preliminares

Hiperparâmetro	Valor / Descrição
policy	MlpPolicy
learning_rate	3×10^{-4}
n_steps	1024
batch_size	64
n_epochs	10
gamma	0,99
gae_lambda	0,95
clip_range	0,2
ent_coef	0,0
device	cuda

Entre os hiperparâmetros variados, `n_steps` demonstrou maior impacto no desempenho, o valor 1024 produziu aprendizado mais estável do que valores menores, pois janelas maiores de coleta fornecem estimativas de vantagem mais precisas antes de cada atualização.

Aumentar `n_epochs` de 10 para 15 não resultou em melhora perceptível, indicando que reutilizar os dados coletados além desse limite não agrega valor na fase exploratória atual. O coeficiente de entropia mantido em zero significa que não há incentivo adicional de exploração via entropia, decisão adequada enquanto a função de recompensa ainda está sendo calibrada.

Esses resultados evidenciam que a sensibilidade ao design da função de recompensa, apontada por Kober et al. [16] como o principal desafio do RL em robótica, se manifesta já nos experimentos preliminares, variações nos hiperparâmetros produziram diferenças de desempenho menores do que as diferenças esperadas com a reformulação da função de recompensa. Esse diagnóstico orienta diretamente a próxima fase do projeto, na qual a função de recompensa será ajustada antes de ampliar o volume de treinamento. A infraestrutura de treinamento validada por esses 10 experimentos está preparada para receber o modelo completo do Atom assim que a integração com o `rcssservermj` for concluída.

Função de Recompensa

A função de recompensa é o componente mais crítico do aprendizado por reforço em robótica. Kober et al. [16] destacam que escolhas inadequadas podem levar o agente a comportamentos indesejados mesmo com algoritmos corretos. No caso deste projeto, a função foi definida para incentivar o avanço e penalizar quedas e esforço excessivo, garantindo que o agente não apenas se mova, mas também mantenha estabilidade e eficiência energética. Sem uma penalidade explícita para quedas, o agente poderia aprender a avançar de forma instável e cair com frequência, o que é inaceitável no contexto competitivo da RoboCup. A função de recompensa pode ser vista abaixo:

$$r_t = w_v \cdot v_x - w_f \cdot \mathcal{K}[\text{queda}] - w_e \cdot \|\tau_t\|^2 \quad (4)$$

onde v_x é a velocidade de avanço do robô no eixo frontal, $\mathcal{K}[\text{queda}]$ é um indicador binário que assume valor 1 quando o robô toca o solo com qualquer parte do corpo que não os pés, $\tau_t \in \mathbb{R}^{20}$ é o vetor de torques aplicados nas 20 articulações no instante t , e w_v , w_f , $w_e \in \mathbb{R}^+$ são pesos escalares que equilibram os três objetivos. A penalidade quadrática $\|\tau_t\|^2$ induz o agente a desenvolver uma caminhada mais eficiente, pois sem ela o agente pode aprender a usar torques elevados que funcionam em simulação, mas que aqueceriam ou danificariam os servomotores Dynamixel em uso prolongado.

Métricas de Avaliação

Quatro métricas principais serão registradas ao longo do treinamento para garantir que a política

aprendida evolui na direção correta, sendo elas a distância percorrida por episódio, a velocidade média de caminhada, a altura do centro de massa e a taxa de quedas [6], [16]. A curva de recompensa total isolada não é suficiente para avaliar a qualidade da política porque um agente pode apresentar recompensa crescente enquanto aprende a permanecer parado em uma postura estável sem avançar. As métricas de distância e velocidade identificam esse comportamento imediatamente, permitindo corrigir a função de recompensa antes de desperdiçar horas de treinamento. A avaliação final ocorrerá em partidas simuladas completas da RoboCup 3D, cenário que MacAlpine e Stone [19] e Abreu et al. [1] identificam como o mais adequado para distinguir políticas de locomoção de boa qualidade.

V. Resultados Preliminares

Os resultados obtidos até o momento cobrem três frentes, sendo elas a validação do ambiente de simulação com o modelo da BahiaRT, a modelagem do Atom e o início dos treinamentos. O treinamento com o Atom ainda está em fase de melhora. Ainda assim, cada etapa concluída reduziu o risco das que virão, pois os problemas foram identificados e resolvidos em um contexto controlado em vez de emergir durante um treinamento longo e de difícil diagnóstico. O progresso até o momento, representa um avanço metodológico concreto e necessário.

A. Validação do Ambiente

A validação do ambiente com o modelo Booster T1 foi a primeira etapa prática do projeto e serviu para isolar potenciais problemas de infraestrutura antes de introduzir a complexidade do Atom. O grupo optou por usar o Booster T1 da equipe BahiaRT [29] porque qualquer problema encontrado poderia ser claramente atribuído ao ambiente ou à configuração do treinamento, e não ao modelo do robô em si. O Booster T1 é o robô principal da BahiaRT na categoria Adult Size, tendo sido amplamente testado em competições internacionais, o que garante que seu comportamento em simulação sirva como referência confiável para detectar anomalias na infraestrutura. Todos os experimentos ocorreram de forma estável e sem instabilidades numéricas perceptíveis, confirmando que a infraestrutura estava pronta para receber o Atom.

Simular uma partida completa no formato Adult Size com os robôs Booster T1 revelou um dado importante sobre a dinâmica do jogo que influenciará diretamente a calibração da função de recompensa. A observação deixou evidente que a recuperação rápida após quedas é muito mais determinante para o desempenho competitivo do que a velocidade de caminhada em campo aberto. Um robô que cai e demora para se levantar perde posicionamento, deixa o adversário livre e frequentemente concede gols, de modo que nenhum ganho de velocidade compensa uma alta taxa de

quedas. Essa observação vai influenciar diretamente a calibração do peso w_f na função de recompensa, que precisará ser alto o suficiente para que o agente aprenda a priorizar o equilíbrio acima de tudo. A Figura 5 ilustra uma das partidas simuladas durante essa etapa, evidenciando o nível de detalhe das interações físicas disponível no ambiente.



Figure 5: Partida simulada no formato Adult Size com os robôs Booster da equipe BahiaRT no simulador oficial da RoboCup 3D, utilizada para validação do ambiente.

B. Modelagem do Atom

A modelagem do Atom avançou de forma consistente, embora com algumas dificuldades maiores do que o previsto, principalmente na etapa de determinar os pesos e as densidades de cada peça. Como essas informações nem sempre estavam disponíveis de forma clara na documentação do robô, foi necessário estimar os materiais utilizados e ajustar manualmente as densidades para que a massa total ficasse próxima da esperada. Essa etapa acabou consumindo mais tempo do que o planejado e revelou um ponto de atenção importante, já que registrar desde o início os parâmetros físicos do hardware é fundamental em projetos que envolvem modelagem física detalhada e a ausência desse registro causou retrabalho considerável.

Partes do modelo já foram exportadas para MJCF e carregadas no simulador para testes de visualização, confirmando que a estrutura cinemática está correta e que o modelo pode ser instanciado sem erros. A próxima etapa consistiu em completar a integração e validar o comportamento físico do modelo, verificando posição do centro de massa, resposta das juntas a torques e estabilidade do contato com o solo, antes de iniciar qualquer treinamento. Iniciar o treinamento com um modelo cujas propriedades físicas não foram validadas pode produzir políticas que funcionam apenas em simulação e são inúteis no hardware real. Um resultado concreto desta fase foi entender em detalhe o processo de inserção de um novo modelo no simulador da RoboCup 3D, que não estava documentado de forma acessível e exigiu estudo cuidadoso do código-fonte, o que vai constituir um recurso valioso para equipes futuras que trabalharemos com a mesma plataforma.

C. Experimentos Preliminares de Treinamento

Para avaliar o comportamento da infraestrutura de treinamento e orientar escolhas para os experimentos

principais com o Atom, foram conduzidos 10 experimentos com PPO utilizando a configuração de rede e hiperparâmetros descritos na Seção IV. Os experimentos variaram sistematicamente $n.steps$, $n.epochs$ e o total de $timesteps$, mantendo os demais hiperparâmetros fixos. A métrica principal de comparação foi a recompensa média obtida ao longo do treinamento, e os resultados completos estão consolidados na Tabela 6. Todos os experimentos foram executados no hardware disponível, um notebook Alienware 16 Aurora com GPU NVIDIA GeForce RTX 3050 6 GB, com oito ambientes paralelos rodando simultaneamente.

Table 6: Resultados dos 10 experimentos preliminares com PPO

Exp.	Timesteps	Ep.	Steps	Rec. Méd.
exp.01	100.000	5	512	-159,57
exp.02	100.000	10	512	-296,07
exp.03	200.000	5	1024	62,15
exp.04	200.000	10	1024	-94,24
exp.05	500.000	5	1024	-6,15
exp.06	500.000	10	1024	63,36
exp.07	1.000.000	5	1024	26,04
exp.08	1.000.000	10	1024	38,33
exp.09	2.000.000	10	1024	96,71
exp.10	2.000.000	15	1024	63,92

Ep. = $n.epochs$; Steps = $n.steps$

O melhor resultado entre os 10 experimentos foi obtido pelo **exp.09**, que atingiu recompensa média de 96,71 e apresentou melhora acentuada da recompensa no início do treinamento seguida de estabilização em valores positivos. A Figura 6 apresenta a recompensa suavizada do exp.09 ao longo dos episódios, representada pela média móvel em janela de 50 episódios, enquanto a Figura 7 apresenta a curva de avaliação com recompensa média e desvio padrão ao longo dos $timesteps$. As oscilações e quedas pontuais observadas em ambas as curvas são esperadas e refletem a variabilidade natural do processo de aprendizagem por reforço. Os demais experimentos apresentaram recompensas médias menores ou curvas de convergência mais instáveis, confirmando que a configuração do exp.09 é o ponto de partida mais adequado para os próximos experimentos.

Os valores positivos de recompensa alcançados pelo exp.09 indicam que o agente aprendeu um comportamento mais eficiente do que o aleatório. O comportamento esperado era que o robô aprendesse a se deslocar para frente mantendo estabilidade, sendo incentivado por uma função de recompensa que considera o avanço no eixo frontal, um bônus por permanecer em pé, um termo de estabilidade relacionado à altura e orientação do tronco, e uma penalidade para comandos muito bruscos. O episódio é encerrado quando o robô apresenta sinais de queda ou instabilidade, como altura do tronco abaixo do limite definido ou inclinação excessiva.

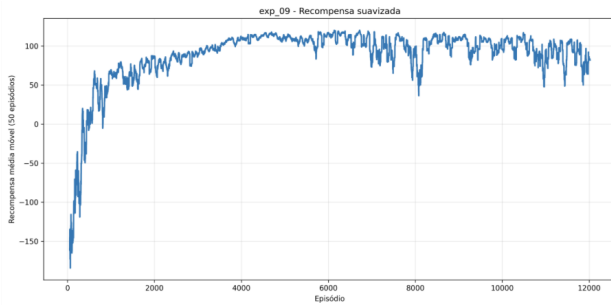


Figure 6: Recompensa suavizada do experimento exp_09 ao longo dos episódios de treinamento. A curva representa a média móvel da recompensa em uma janela de 50 episódios, mostrando melhora acentuada no início do treinamento seguida de estabilização em valores positivos.

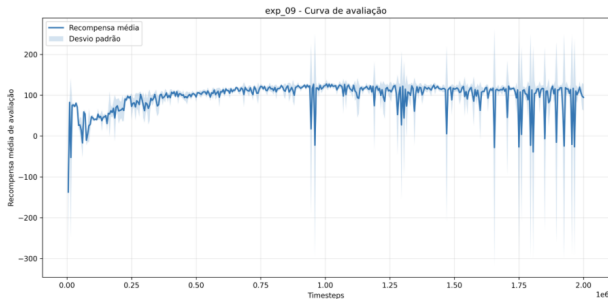


Figure 7: Curva de avaliação do experimento exp_09 durante o treinamento com PPO. A linha azul indica a recompensa média obtida pelo agente nas avaliações ao longo dos *timesteps*, e a região azul clara representa o desvio padrão. Observa-se aumento progressivo da recompensa no início do treinamento e posterior estabilização em valores positivos.

Os resultados iniciais demonstraram que o código de treinamento encontra-se funcional, seguindo uma direção promissora e que a próxima prioridade é reformular a função de recompensa para que o agente seja incentivado a desenvolver padrões de caminhada estáveis e eficientes, condição necessária para que os experimentos com o modelo completo do Atom produzam resultados competitivamente relevantes.

VI. Discussão

Os resultados preliminares indicam que o projeto está avançando de forma consistente com o planejamento inicial, e as decisões metodológicas adotadas até o momento se mostraram adequadas. Trabalhar em paralelo com o modelo da BahiaRT e com a modelagem do Atom foi uma escolha importante para manter o progresso do projeto. Ao testar conceitos com um modelo já existente enquanto o Atom ainda estava sendo desenvolvido, o grupo conseguiu identificar possíveis problemas de integração e entender o funcionamento do sistema de forma antecipada. Essa abordagem reduziu incertezas porque os problemas foram encontrados em um contexto mais controlado, em vez de aparecerem durante fases mais críticas.

A utilização do modelo da BahiaRT como referência

prática também facilitou o aprendizado técnico da equipe. Analisar o funcionamento desse modelo permitiu observar como os dados de percepção são processados e como o controle de locomoção pode ser estruturado dentro do ambiente de simulação. Com isso, o grupo tomou decisões mais informadas durante a construção do Atom, evitando retrabalho e alinhando as escolhas técnicas com soluções que já demonstraram funcionar em contextos semelhantes.

Essa estratégia também contribuiu para o cumprimento do cronograma. Como diferentes frentes evoluíram ao mesmo tempo, o grupo conseguiu manter um ritmo constante de desenvolvimento mesmo enquanto partes da modelagem do Atom ainda estavam em andamento. O paralelismo nas atividades se refletiu diretamente no estado atual do projeto, e por isso essa organização em paralelo é uma prática metodológica que vale manter nas fases seguintes.

Os experimentos e o desenvolvimento realizados até o momento foram conduzidos em um notebook Alienware 16 Aurora AC16250, equipado com processador Intel® Core™ 7 240H com 16 núcleos, 16 GB de memória RAM e uma GPU dedicada NVIDIA GeForce RTX 3050 Laptop GPU com 6 GB de memória de vídeo, sob o sistema operacional Ubuntu 24.04.1 LTS. Os oito ambientes de simulação paralelos foram executados nessa configuração durante a validação com o modelo da BahiaRT sem uso excessivo de memória ou instabilidades perceptíveis. Esse resultado elimina o risco de que limitações de hardware comprometam o cronograma, permitindo que o foco das próximas etapas seja inteiramente na qualidade do modelo e da função de recompensa.

A principal incerteza que ainda permanece diz respeito à qualidade do modelo físico do Atom para o treinamento. Mesmo com todos os cuidados adotados, o modelo MJCF é uma aproximação, já que geometrias de colisão são simplificadas, atuadores são tratados como motores ideais e efeitos como folga mecânica e atrito nas juntas não são completamente capturados, o que caracteriza o *sim-to-real gap* [14]. Para o escopo deste trabalho, o objetivo não é resolver esse *gap* completamente, mas produzir uma política competitiva em simulação como prova de conceito, deixando a transferência para hardware como trabalho futuro. Uma forma direta de verificar a qualidade do modelo antes do treinamento é comparar as frequências naturais de oscilação do robô físico com as do modelo simulado, pois se diferirem significativamente os parâmetros de massa ou inércia precisam ser revisados antes de iniciar o treinamento principal.

A *domain randomization* é a técnica mais promissora para reduzir o *sim-to-real gap* em trabalhos futuros com o Atom. Ela consiste em variar aleatoriamente parâmetros do modelo durante o treinamento, como massas, coeficientes de atrito e atrasos nos atuadores, tornando a política aprendida mais robusta a variações [14]. Peng et al. [24] mostram que combi-

nar *domain randomization* com referências de movimento melhora ainda mais a qualidade das políticas aprendidas, indicando que essa extensão tem potencial relevante para o Atom. Por esse motivo, a infraestrutura de treinamento está sendo construída de forma a facilitar essa extensão futura.

Encontrar bons valores para os pesos da função de recompensa é outro aspecto que precisará de atenção dedicada nas primeiras semanas de treinamento. Valores muito altos para w_f levam o agente a permanecer parado para evitar quedas, enquanto valores muito baixos para w_e resultam em uma caminhada funcionalmente correta, mas com torques que danificariam os servomotores em uso prolongado. Como alertam Kober et al. [16], esse ajuste é inevitável em RL para robótica e precisa ser tratado como parte central da metodologia, não como detalhe de implementação, e por isso tempo explícito foi reservado no cronograma para essa fase de calibração.

Em relação às limitações computacionais, o treinamento do agente exige um volume considerável de simulação para alcançar bons resultados. Pelas estimativas iniciais, serão necessárias de 2 a 4 semanas de treinamento contínuo para atingir uma convergência satisfatória, período que já foi considerado no cronograma. A paralelização com ambientes simultâneos é fundamental para tornar esse processo viável, e a arquitetura cliente-servidor do *rcssservermj* permite executar esses ambientes em paralelo sem modificações no servidor. Isso confirma que a infraestrutura escolhida está adequada para o volume de treinamento necessário.

Do ponto de vista das contribuições para a RoboFEI, o projeto também gera resultados que continuam úteis independentemente dos números finais do treinamento. O modelo MJCF calibrado do Atom, que exigiu um trabalho extenso de medição, montagem e ajuste, pode servir como base para projetos futuros que envolvam simulação do robô. A documentação do processo de inserção de novos modelos no servidor oficial da RoboCup 3D, que não estava disponível de forma clara e precisou ser reconstruída a partir do código-fonte, pode facilitar o trabalho de futuros integrantes da equipe. Esses resultados têm valor independente do desempenho final do agente, e por isso são contribuições importantes para a continuidade da equipe.

Além dos resultados técnicos, o projeto abre caminho para uma nova frente de atuação da equipe, o futebol de robôs em simulação 3D. Ao desenvolver o modelo do robô, a infraestrutura de treinamento e a integração com o ambiente de simulação, o trabalho cria a base necessária para que a RoboFEI possa explorar essa categoria no futuro. Isso amplia as possibilidades de pesquisa da equipe e permite testar estratégias, algoritmos e comportamentos complexos sem depender exclusivamente de experimentos com o robô físico. O projeto não contribui apenas com resultados, mas

também cria uma base técnica importante para que a equipe consiga participar da categoria de futebol de robôs em simulação 3D nos próximos anos.

VII. Cronograma

A Tabela 7 apresenta o cronograma previsto para as etapas restantes até novembro de 2026. As atividades foram organizadas respeitando as dependências entre si, e a sobreposição entre a redação da monografia e as demais atividades é intencional, já que documentar os resultados à medida que surgem evita a concentração de escrita no final do período, quando detalhes importantes tendem a ser esquecidos. Essa estrutura mostra uma estratégia pensada para produzir conhecimento durante todo o projeto, e não somente no final.

Table 7: Cronograma para 2026

Atividade	Jan	Fev	Mar	Abr	Mai	Jun	Jul	Ago	Set	Out	Nov
1. Início das pesquisas	•										
2. Def. observações	•	•									
3. Teste do simulador			•	•							
4. Revisão bibliográfica		•	•	•	•	•					
5. Integração modelo Atom		•	•	•	•						
6. Treinamento PPO					•	•	•	•			
7. Avaliação em partidas								•	•		
8. Redação da monografia							•	•	•	•	
9. Entrega/apresentação											•

VIII. Referências

- [1] M. Abreu, N. Lau e L. P. Reis, “Learning Low-Level Motor Skills for Soccer Playing Humanoid Robots,” em *IEEE ICARSC*, Santa Maria da Feira, Portugal, 2021, pp. 1–8.
- [2] M. Abreu, N. Lau, A. Sousa e L. Reis, “Learning Low-Level Skills from Scratch for Humanoid Robot Soccer Using Deep Reinforcement Learning,” em *IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, Porto, Portugal, 2019, pp. 1–8.
- [3] J. Allali et al., “Rhoban Football Club 2023 Champion Team Paper,” em *RoboCup 2023*, Cham: Springer, 2024.
- [4] G. Brockman et al. “OpenAI Gym,” acessado em 25 de mar. de 2026. URL: <https://arxiv.org/abs/1606.01540>.
- [5] E. Coumans e Y. Bai, *PyBullet: Physics Simulation for Robotics*, 2021. acessado em 2 de abr. de 2026. URL: <http://pybullet.org>.
- [6] Y. Duan, X. Chen, R. Houthooft, J. Schulman e P. Abbeel, “Benchmarking Deep Reinforcement Learning for Continuous Control,” em *International Conference on Machine Learning (ICML)*, New York, NY, 2016, pp. 1329–1338.
- [7] S. Fujimoto, H. van Hoof e D. Meger, “Addressing Function Approximation Error in Actor-Critic Methods,” em *International Conference on Machine Learning (ICML)*, Stockholm, Sweden, 2018, pp. 1587–1596.
- [8] F. Gianzanti. “op3_model e op3_trainner.” Disponível também em: https://github.com/Gianzanti/op3_trainner, acessado em 10 de abr. de 2026. URL: https://github.com/Gianzanti/op3_model.
- [9] R. F. Gonçalves, “Deep Reinforcement Learning Application in Humanoid Robots Locomotion,” tese de mestrado, Universidade Estadual de Campinas, Campinas, SP, 2021.
- [10] L. Gondry et al., “Rhoban Football Club 2019 Champion Team Paper,” em *RoboCup 2019*, Cham: Springer, 2019, pp. 491–503.
- [11] S. Ha et al., “Learning-Based Legged Locomotion: State of the Art and Future Perspectives,” *The International Journal of Robotics Research*, vol. 44, n.º 1, pp. 1–43, 2025.
- [12] T. Haarnoja, A. Zhou, P. Abbeel e S. Levine. “Learning to Walk via Deep Reinforcement Learning,” acessado em 12 de mar. de 2026. URL: <https://arxiv.org/abs/1812.11103>.
- [13] N. Heess et al. “Emergence of Locomotion Behaviours in Rich Environments,” acessado em 22 de mar. de 2026. URL: <https://arxiv.org/abs/1707.02286>.
- [14] J. Hwangbo et al., “Learning Agile and Dynamic Motor Skills for Legged Robots,” *Science Robotics*, vol. 4, n.º 26, eaau5872, 2019.
- [15] H. Kitano, M. Asada, Y. Kuniyoshi, I. Noda e E. Osawa, “RoboCup: The Robot World Cup Initiative,” em *International Conference on Autonomous Agents*, Marina del Rey, CA, 1997, pp. 340–347.
- [16] J. Kober, A. Bagnell e J. Peters, “Reinforcement Learning in Robotics: A Survey,” *The International Journal of Robotics Research*, vol. 32, n.º 11, pp. 1238–1274, 2013.
- [17] Y. LeCun, Y. Bengio e G. Hinton, “Deep Learning,” *Nature*, vol. 521, n.º 7553, pp. 436–444, 2015.
- [18] T. P. Lillicrap et al., “Continuous Control with Deep Reinforcement Learning,” em *International Conference on Learning Representations (ICLR)*, 2016.
- [19] P. MacAlpine e P. Stone, “UT Austin Villa: RoboCup 3D Simulation League Champions,” em *RoboCup 2017: Robot World Cup XXI*, Cham: Springer, 2018, pp. 473–485.

- [20] M. R. O. A. Maximo e C. H. C. Ribeiro, “Omni-directional Zero Moment Point Walking Biped Robot,” *Journal of Control, Automation and Electrical Systems*, vol. 30, n.º 4, pp. 519–531, 2019.
- [21] D. Melo, M. R. O. A. Maximo e A. Cunha, “Learning Push Recovery Behaviors for Humanoid Walking Using Deep Reinforcement Learning,” *Journal of Intelligent and Robotic Systems*, vol. 105, n.º 2, pp. 1–18, 2022.
- [22] V. Mnih et al., “Human-Level Control through Deep Reinforcement Learning,” *Nature*, vol. 518, n.º 7540, pp. 529–533, 2015.
- [23] U. F. de Ouro Preto. “Fundamentos de Redes Neurais,” acessado em 5 de abr. de 2026. URL: <https://www2.decom.ufop.br/imobilis/fundamentos-de-redes-neurais/>.
- [24] X. B. Peng et al., “AMP: Adversarial Motion Priors for Stylized Physics-Based Character Control,” em *ACM SIGGRAPH*, 2021.
- [25] J. Peters, S. Vijayakumar e S. Schaal, “Reinforcement Learning for Humanoid Robotics,” em *IEEE-RAS International Conference on Humanoid Robots*, Karlsruhe, Germany, 2003, pp. 1–20.
- [26] J. Schulman et al., “Trust Region Policy Optimization,” em *International Conference on Machine Learning (ICML)*, Lille, France, 2015, pp. 1889–1897.
- [27] J. Schulman, F. Wolski, P. Dhariwal, A. Radford e O. Klimov. “Proximal Policy Optimization Algorithms,” acessado em 18 de mar. de 2026. URL: <https://arxiv.org/abs/1707.06347>.
- [28] Q. Shi, Y. Zhang e Z. Li, “Deep Reinforcement Learning-Based Attitude Motion Control for Humanoid Robots with Stable Training Strategies,” *Industrial Robot: The International Journal of Robotics Research and Application*, vol. 47, n.º 5, pp. 741–752, 2020.
- [29] M. Simões et al., “BahiaRT 2022: Team Description Paper,” em *RoboCup 2022*, Bangkok, Thailand, 2022.
- [30] R. S. Sutton e A. G. Barto, *Reinforcement Learning: An Introduction*, 2ª ed. Cambridge, MA: MIT Press, 2018.
- [31] E. Todorov, T. Erez e Y. Tassa, “MuJoCo: A Physics Engine for Model-Based Control,” em *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vilamoura, Portugal, 2012, pp. 5026–5033.
- [32] M. Vukobratović e B. Borovac, “Zero-Moment Point — Thirty Five Years of Its Life,” *International Journal of Humanoid Robotics*, vol. 1, n.º 1, pp. 157–173, 2004.
- [33] C. J. C. H. Watkins e P. Dayan, “Q-Learning,” *Machine Learning*, vol. 8, n.º 3–4, pp. 279–292, 1992.